Classes and instances

Today

Welcome to your next lesson: Classes and instances.

I'm preparing your personalized lesson based on what you've already learned, what's next, and the best ways to help you build your skills.

Step by step, you’re moving forward.

Ron, let’s build on what you just did with objects = data + behavior 🧠

Imagine a pet shelter system. Instead of thinking about one specific dog or cat, you first design a general pet description:

It says every pet has a name, age, and type (dog, cat, etc.)
It says every pet can do things like eat, sleep, and play

This general design is what we call a class — it’s like a blueprint for pets, not a real pet yet.

Quick check-in before we go deeper:

Does the idea “a class is like a blueprint or general design for a kind of object” feel clear so far, or a bit fuzzy?

That makes sense

Nice 👌

Let’s add the next piece: instances.

Go back to the shelter:

Pet = the blueprint (general design)
Luna, Max, and Pepper = three actual animals in the shelter

In programming terms:

A class is the blueprint (Pet),

an instance is one real object made from that blueprint (Luna, Max, Pepper).

So: one Pet class → many pet instances.

Does that mapping feel right to you:

Pet = class / blueprint, and Luna = instance / real object?

Yes, that makes sense

Here’s a visual to lock it in 🐶🐱👇

Top card: Pet = class / blueprint (shared structure & behaviors)
Bottom cards: luna, max, pepper = instances (real pets made from that blueprint)
Look at the diagram for a moment.

Then tell me: in your own words, what is the one thing the top Pet card represents, and what are the bottom cards?

Class vs instances: Pet, Luna, Max

the top card represents the generic outline of all pets the botton card represents an actual pet using top cards bluprint format

What do you think? Take your time with the question below 👇

What does the Pet card represent in our diagram, compared to luna and max?

A.
Pet is the blueprint, and luna / max are real pets made from

it.

B.
Pet is one specific pet, and luna / max are blueprints.

C.
They are all blueprints, none are real pets.

D.
They are all individual pets, none is a blueprint.

Exactly right 🎯

Pet is the blueprint — it describes what all pets have and can do.
luna and max are instances — real, individual pets created from that Pet blueprint.

So the class = kind of thing, and the instances = which specific thing.

One more small idea, then we'll connect it to Python.

In Python, we usually write:

Class names in CapitalCase, like Pet
Instance (object) names in snake_case, like luna or my_pet

Does that naming difference feel easy to remember so far? 🐾

Yes, that feels easy

Great 😄

Let's connect this to something you already know in Python: lists.

Think of:

list → like the class / blueprint (the general idea of “a list”)
numbers = [1, 2, 3] → one instance (one actual list)
names = ["Ron", "Alex"] → another instance (a different list)

So just like:

Pet → blueprint
luna, max → instances

We also have:

list → blueprint
numbers, names → instances
Does that parallel between Pet/pets and list/actual lists feel clear?

Yes, that makes sense

What do you think? 👇

A pet shelter uses one Pet blueprint to describe Luna, Max, and Pepper. What are Luna, Max, and Pepper?

A.
Three classes

B.
Three instances

C.
One naming convention

D.
One blueprint

Nice work, Ron 🎉 You nailed it.

Luna, Max, and Pepper are three instances because:

Pet is the one blueprint (class) that describes what any pet is like.
Each of Luna, Max, and Pepper is one individual object created from that blueprint.

So: 1 class → many instances. You've got the core idea of classes vs. instances down. 💪

You now understand:

Class = blueprint (Pet)
Instance = real object made from that blueprint (luna, max, pepper)
Class names in CapitalCase, instances in snake_case

Want to move on to the next lesson now, or do you want one more quick example to reinforce this?

Let's move on

Love it, Ron — moving forward with momentum 🚀

Quick recap of what you've just mastered:

A class is a blueprint (like Pet) that describes shared structure and behavior
An instance is a real object made from that blueprint (like luna, max, pepper)
One class → many instances
In Python, class names use CapitalCase (e.g. Pet), instance names use snake_case (e.g. my_pet)
The same idea shows up with things you already know, like list vs numbers = [1, 2, 3]

You're thinking in objects now, not just in data and functions. That's a big shift, and you handled it well 💪

When you're ready, hit Next lesson in the Learn tab, and I'll be right there with you for the next step.